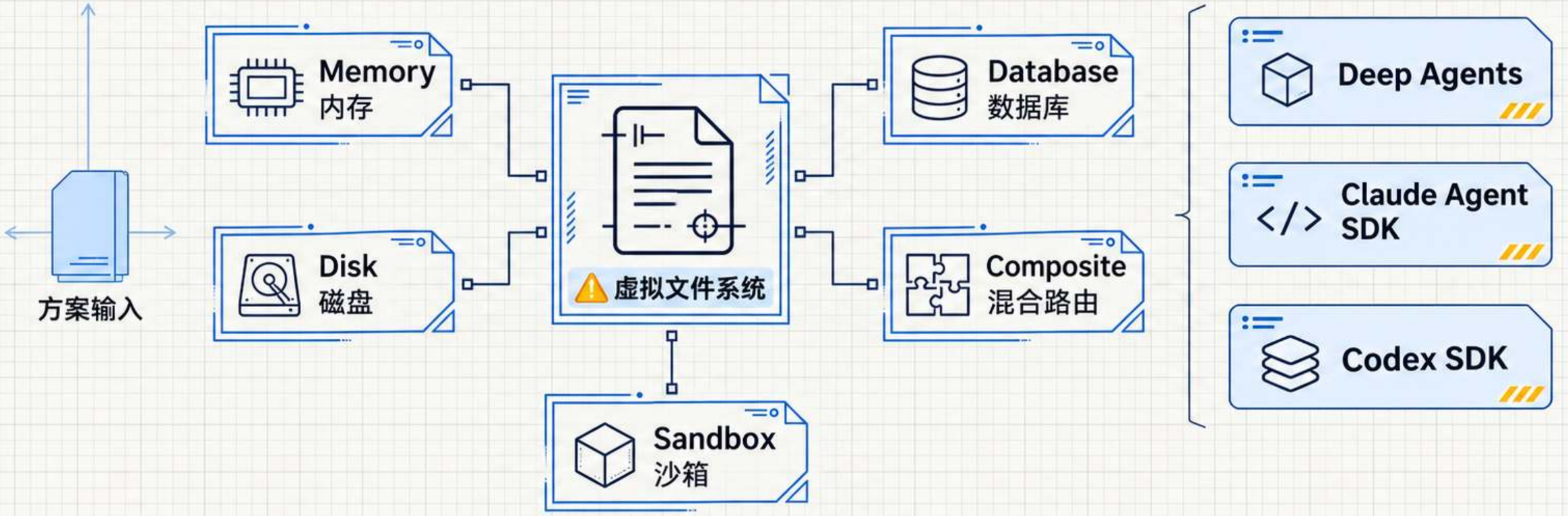
Deep Agents 实战

第1章·下篇：Context Engineering 与方案对比



上一讲回顾：Agent 开发的三个层次

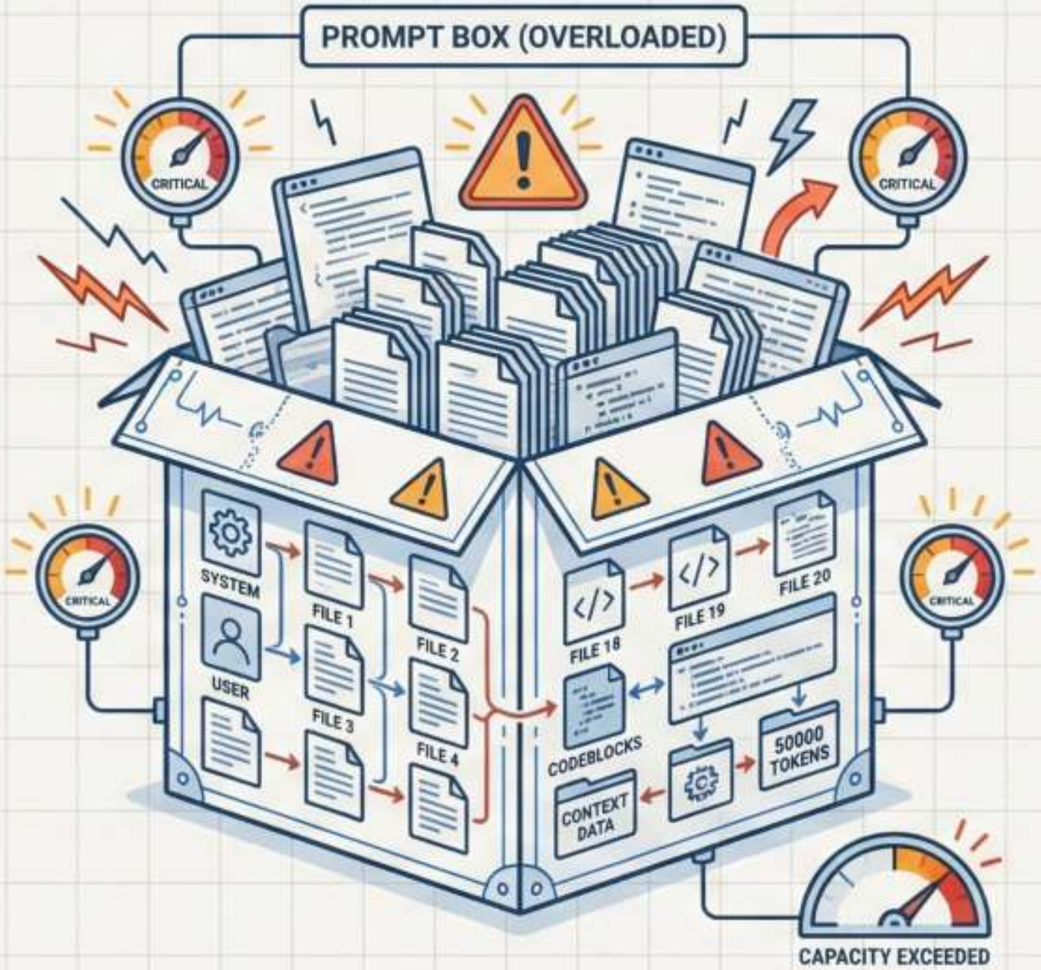
Runtime → Framework → Harness



今天的问题：Harness 的核心理念是什么？与竞品有何不同？

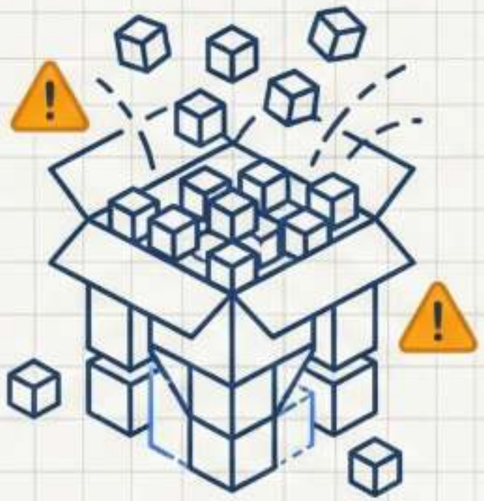
传统做法：Prompt Stuffing

把所有信息塞进提示词

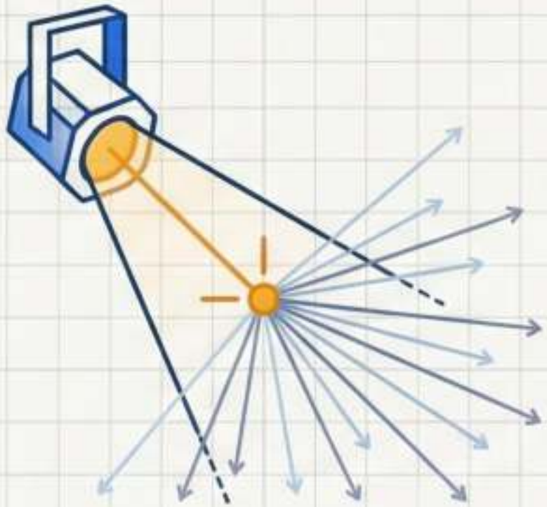


- System：你是一个编程助手
- User：请帮我重构 src/ 下的代码
- [附带：20 个文件的完整内容，共 50000 tokens]
- 结果：LLM 被迫一次性消化所有信息

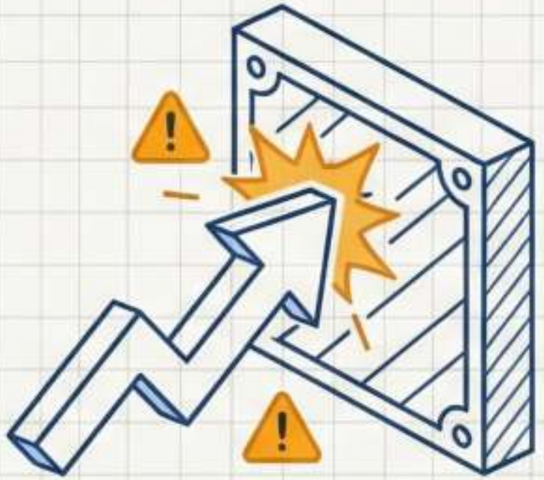
Prompt Stuffing 的三个致命问题



上下文窗口溢出：LLM 有 token 上限，文件一多就装不下



注意力稀释：信息越多，LLM 对关键信息的关注度越低



不可扩展：无法处理任意规模的项目

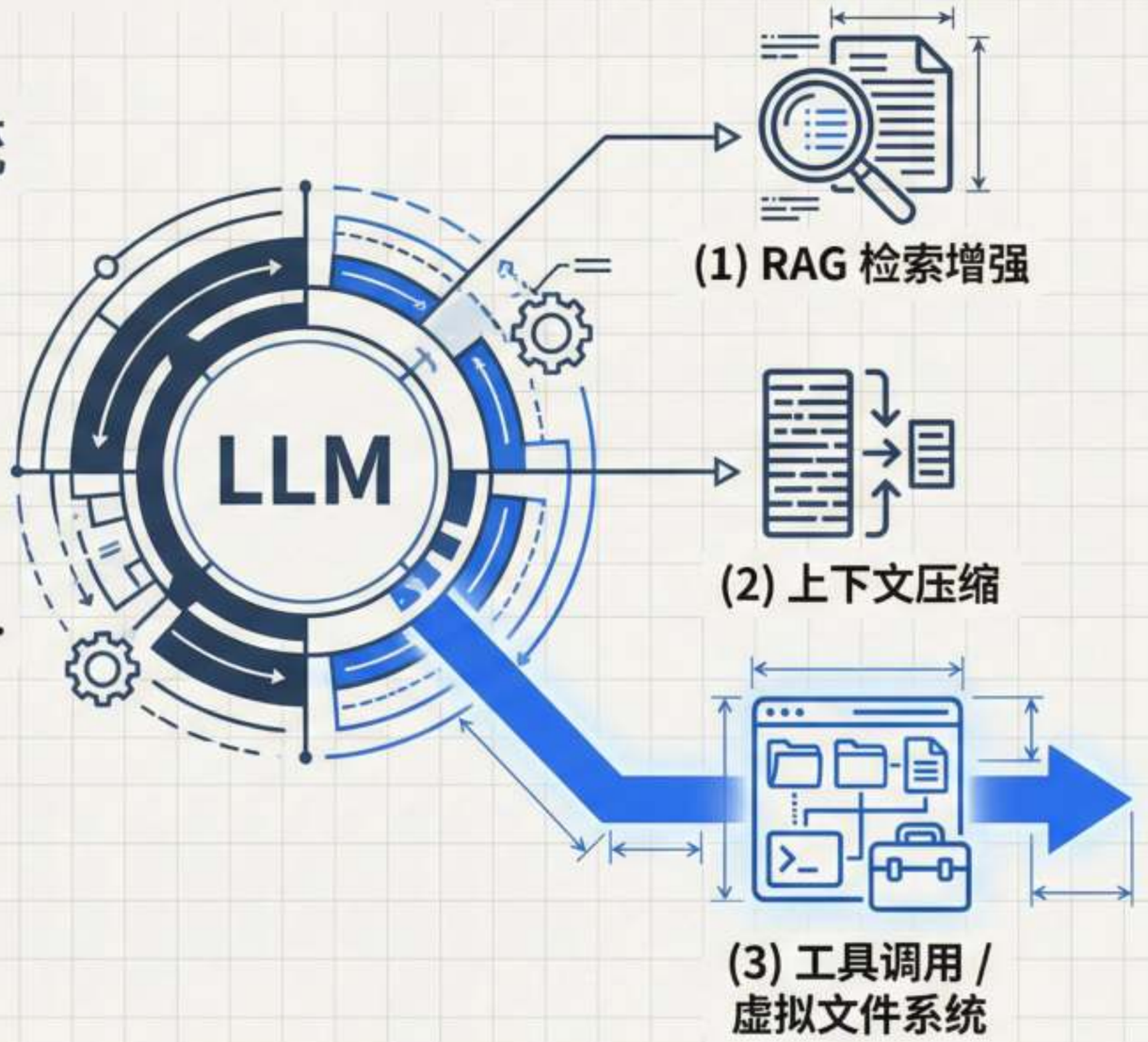
Context Engineering：上下文工程

围绕 LLM 构建高效获取信息的系统

- 核心思想：不是把信息塞给 LLM，
  而是让 LLM 能按需获取信息

- 不同产品有不同的实现策略
- RAG 检索增强 / 上下文压缩 / 工具调用 …

- Deep Agents 的选择 → **虚拟文件系统**

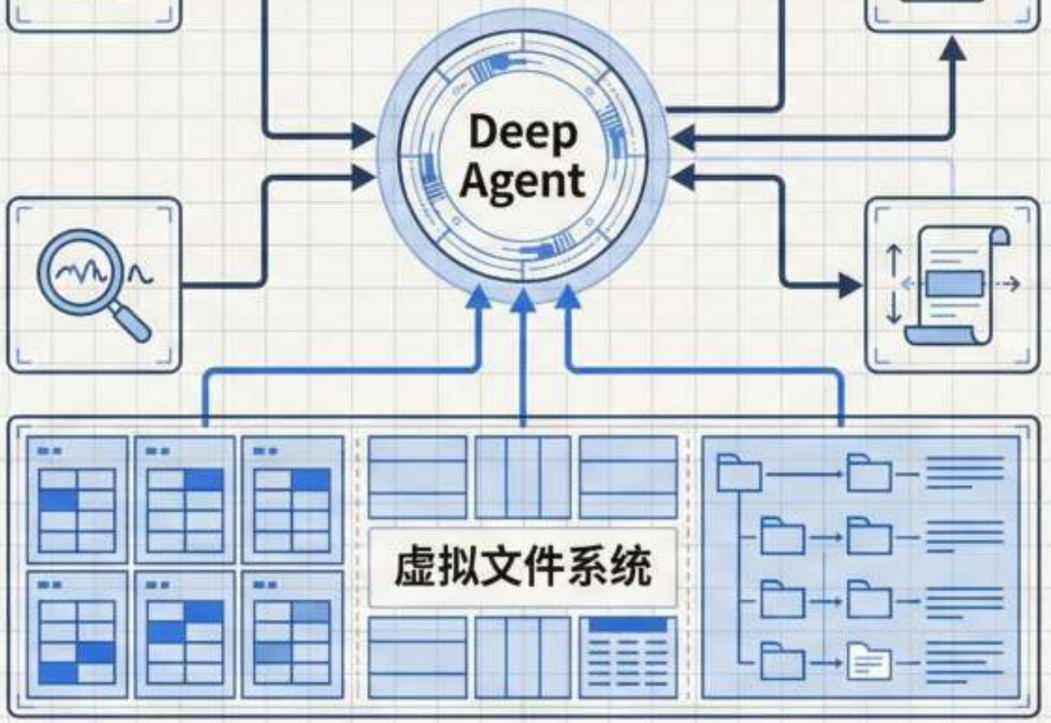


Deep Agents 的解法：虚拟文件系统

让 Agent 像人类一样按需获取信息

read_file
— 按需读取文件内容

write_file
— 记录中间结果



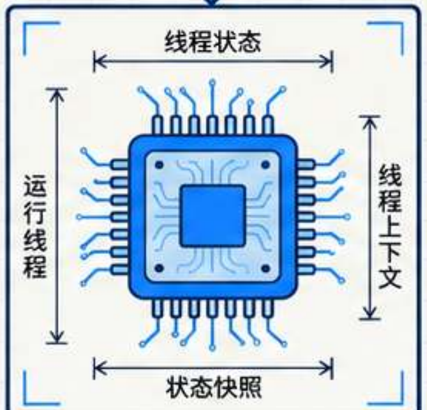
grep / glob
— 搜索定位信息

offset / limit
— 大文件只读取需要的部分

核心理念：上下文里只保留当前步骤真正需要的信息

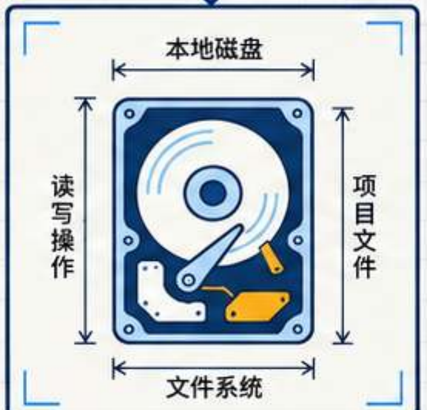
可插拔后端：统一接口，多种实现

统一接口 API



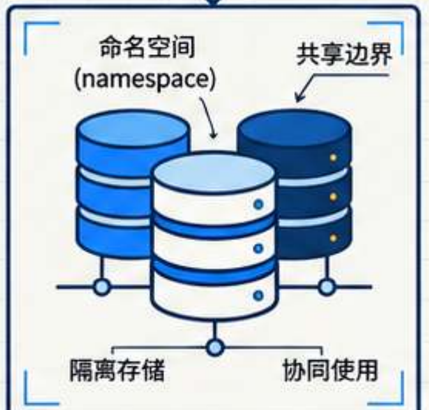
线程状态
StateBackend

工作空间保存在 Agent State



本地磁盘
FilesystemBackend

读写真实项目文件



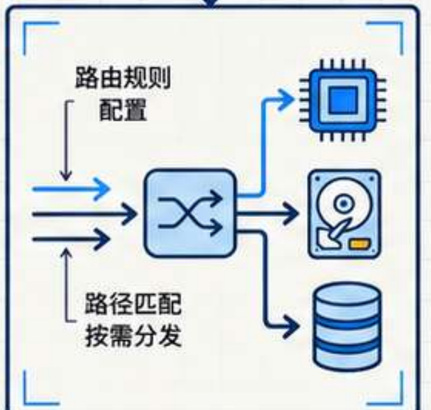
跨线程存储
StoreBackend

namespace 划分
共享边界



隔离执行
沙箱后端

通过 Backend
接入执行环境



混合路由
CompositeBackend

不同路径使用不同后端

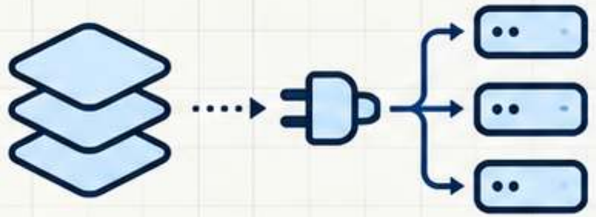
⚠ | 持久化取决于 checkpointer 或 Store 的实际实现。

⚠ | 沙箱边界依赖配置；本地 Shell 不等于沙箱。

三种 Agent Harness：定位与接入

Deep Agents

通用、可组合的
Agent Harness



多模型接口与可插拔 Backend

Python · TypeScript

Claude Agent SDK

将 Claude Code
能力嵌入应用



工具、会话、权限与 Hooks

Python · TypeScript

Codex SDK

在应用中编程控制
Codex



本节聚焦 TypeScript SDK

封装本地 Codex CLI

⚠ | **按架构与运行方式比较，不把功能相似误写成完全等同。**

核心差异：能力如何装配与运行

Deep Agents

Deep Agents 的能力装配与运行

- 多模型与 Middleware 组合
- 虚拟文件系统 + 可插拔 Backend
- Store 支持跨线程数据
- 沙箱作为执行后端
- 配合 LangSmith 观测

Claude Agent SDK

Claude Agent SDK 的能力装配与运行

- Claude Code 的工具与循环
- Hooks 与权限控制
- 会话、Skills 与子 Agent
- 应用自行部署运行进程

Codex SDK

Codex SDK 的能力装配与运行

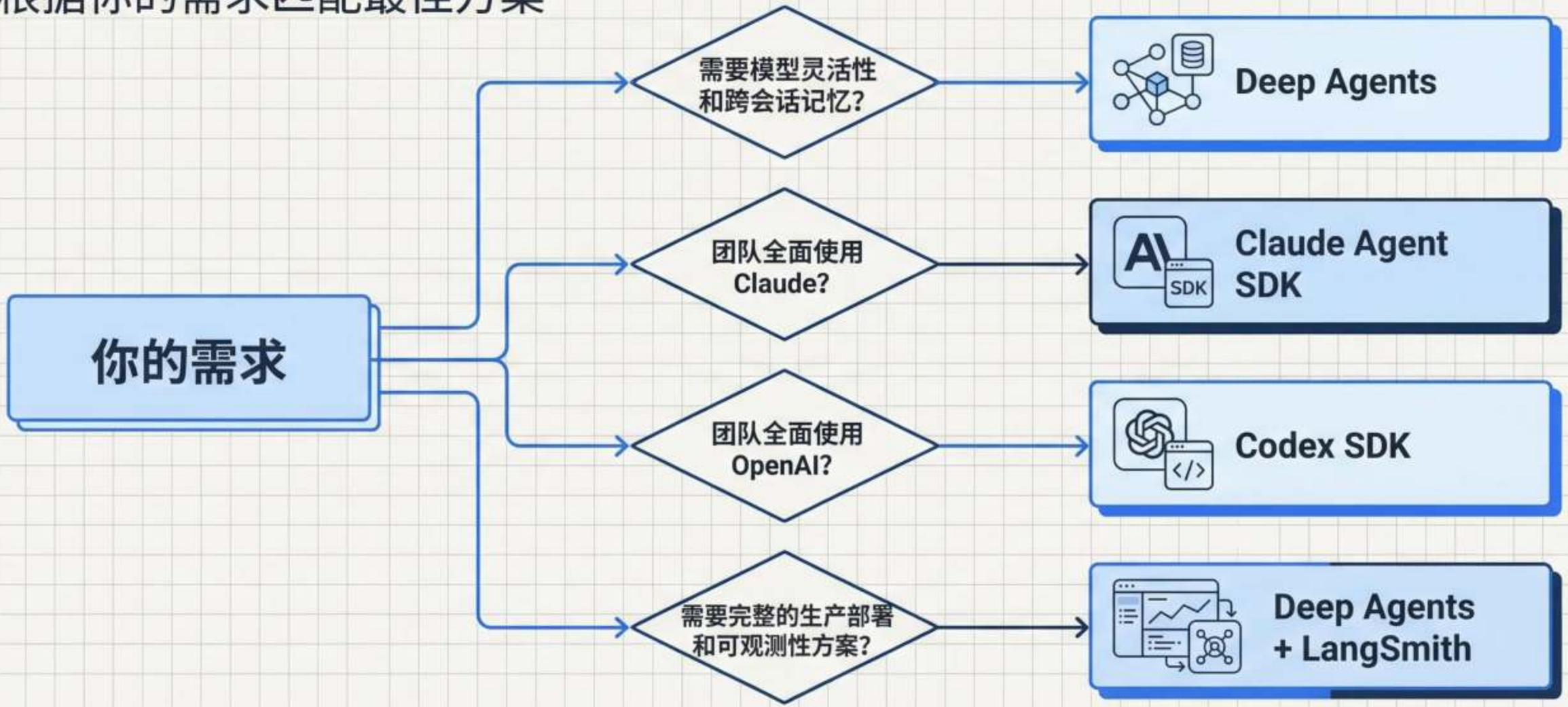
- TypeScript SDK 封装本地 CLI
- 可配置审批与沙箱策略
- 可接入 MCP 工具

⚠ | **托管云端运行：Agents API，与本地 Codex SDK 分开理解。**

⚠ | 以上是架构侧重点，不是排他功能清单；具体能力取决于版本与配置。

如何选择?

根据你的需求匹配最佳方案



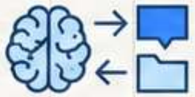
本篇回顾



- Context Engineering：为 LLM 组织按需获取信息的能力。



- 虚拟文件系统与可插拔 Backend：分离接口和存储、执行环境。

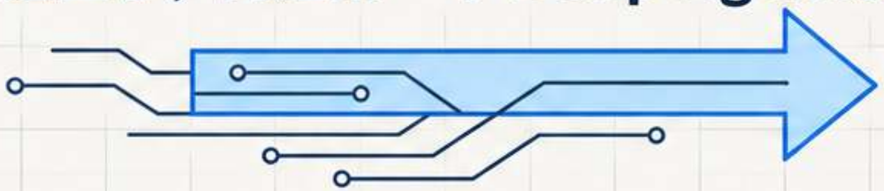


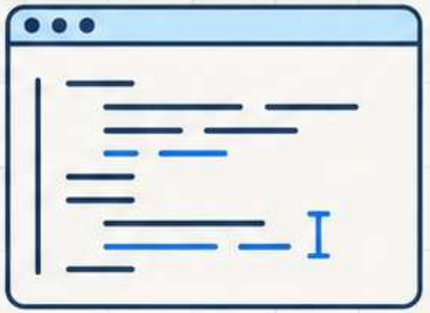
- 选型看组合方式、运行责任、持久化与权限边界。



- 区分本地 Codex SDK 与托管 Agents API。

下一章：快速上手，构建第一个 Deep Agent。





⚠ | 对比是架构参考，实际支持以官方文档和所用版本为准。